

CAHIER DES CHARGES

Projet de fin d'études

Plateforme Cloud Native Lakehouse pour l'analyse du marché de l'emploi tech

Auteur	Nouhayla YOUSFI
Encadrant	Prof NAJI Abdelwahab
Établissement	École nationale des sciences appliquées - Oujda
Filière / Spécialité	Data Science & Cloud Computing
Version	1.0
Date	4 juin 2026

Remarque : Ce document ne décrit pas exactement le déroulement du projet ; il s'agit simplement d'une planification prévisionnelle établie à l'avance.

1. Description de l'état actuel

1.1 Contexte général

Le marché de l'emploi dans le secteur technologique évolue à une vitesse croissante. Les compétences recherchées par les entreprises se transforment rapidement sous l'effet de l'adoption massive du cloud, de l'intelligence artificielle et de l'automatisation. Cette dynamique crée un décalage croissant entre les compétences disponibles sur le marché et celles effectivement demandées par les recruteurs.

Dans ce contexte, les acteurs de la formation professionnelle, les étudiants, les chercheurs d'emploi et les décideurs RH manquent d'outils capables de leur fournir une vision en temps quasi-réel, multi-régionale et actionnable des tendances de compétences tech.

1.2 Problématique

Les analyses du marché de l'emploi tech disponibles aujourd'hui souffrent de plusieurs limites structurelles :

- Les données sont fragmentées entre de multiples plateformes (LinkedIn, Indeed, WTTJ, Rekrute...) sans consolidation unifiée.
- Les rapports de tendances sont produits manuellement, avec une fréquence insuffisante (trimestrielle ou annuelle) et une granularité géographique limitée.
- Il n'existe pas de système automatisé capable de détecter en continu l'émergence ou le déclin de compétences spécifiques par région ou secteur.
- Les décideurs (formateurs, RH, politiques publiques) ne disposent pas d'insights intelligents générés automatiquement pour orienter leurs stratégies.

2. Objectifs du projet

2.1 Objectif principal

Concevoir et implémenter une plateforme Cloud Native Lakehouse capable de collecter, traiter et analyser en continu les données du marché de l'emploi tech dans plusieurs régions géographiques, afin de détecter automatiquement les tendances de compétences et de générer des insights intelligents via des agents d'intelligence artificielle.

2.2 Objectifs spécifiques

- Automatiser la collecte de données depuis les principales plateformes d'offres d'emploi tech au Maroc et en France.
- Mettre en place un pipeline de streaming temps réel pour l'ingération et la validation des données brutes.
- Implémenter une architecture Lakehouse en couches (Bronze / Silver / Gold) garantissant la qualité, la traçabilité et la disponibilité des données.
- Déployer des agents IA capables de détecter les tendances émergentes, générer des rapports automatiques et répondre à des requêtes analytiques en langage naturel.
- Exposer les résultats via des tableaux de bord interactifs accessibles aux utilisateurs finaux.
- Déployer l'ensemble de la plateforme sur Kubernetes selon les principes Cloud Native, avec une architecture conçue pour être compatible avec Databricks.

2.3 Périmètre géographique

La phase initiale du projet couvre deux zones géographiques :

- Le Maroc : avec un focus sur les principales métropoles (Casablanca, Rabat, Marrakech, Agadir).
- La France : avec une couverture nationale (Paris, Lyon, Bordeaux, Toulouse, Nantes, Lille).

L'architecture du système est conçue de façon modulaire afin de permettre l'extension à d'autres marchés (Belgique, Canada, Afrique francophone) sans refonte majeure.

3. Périmètre du projet

3.1 Ce qui est inclus dans le projet

3.1.1 Collecte de données

- Scraping automatisé des offres d'emploi sur les plateformes : Welcome to the Jungle (France), Rekrute (Maroc), Emploi.ma, Indeed.
- Consommation d'APIs publiques disponibles (Adzuna API, API Emploi de Pole Emploi).
- Gestion des proxies et rotation d'identité pour éviter les blocages.
- Collecte des champs : titre du poste, compétences requises, localisation, niveau d'expérience, type de contrat, salaire (si disponible), date de publication, entreprise.

3.1.2 Pipeline de données

- Ingération en streaming via Apache Kafka.
- Validation de schéma à l'entrée des topics.
- Stockage en couches Bronze (brut), Silver (nettoyé), Gold (agrégé) sur Delta Lake.
- Transformations SQL gérées via dbt.
- Traitement distribué avec Apache Spark / PySpark.

3.1.3 Analyse et IA

- Extraction de compétences depuis le texte libre .
- Détection de tendances : émergence ou déclin de compétences, variations géographiques.
- Agents IA capables de répondre à des questions analytiques sur les données.

3.1.4 Exposition des données

- Query layer avec Trino ou DuckDB pour les requêtes analytiques.
- Tableaux de bord interactifs avec Apache Superset ou Grafana.
- API REST pour l'interrogation programmatique des données Gold.

3.1.5 Infrastructure

- Déploiement complet sur Kubernetes (Helm charts).
- CI/CD avec GitHub Actions.
- Monitoring de l'infrastructure et des pipelines.

3.2 Ce qui est exclu du projet

- Développement d'une application mobile ou d'une interface web custom (les dashboards suffisent).
- Couverture de marchés hors Maroc et France pour la phase initiale.
- Gestion des données personnelles des candidats (seules les offres d'emploi sont traitées).
- Intégration avec des systèmes RH tiers (ATS, SIRH).

4. Spécifications fonctionnelles

4.1 Modules fonctionnels

4.1.1 Module de collecte

Le système doit être capable de :

- Lancer des collectes planifiées (quotidiennes ou à la demande) sur les sources configurées.
- Détecter et éviter la duplication des offres déjà collectées (déduplication par hash).
- Signaler les échecs de collecte et relancer automatiquement.
- Supporter l'ajout de nouvelles sources sans modification du cœur du pipeline.

4.1.2 Module d'ingération et validation

- Valider la structure des données entrantes avant écriture en Bronze.
- Enrichir chaque enregistrement avec : horodatage d'ingération, source, pays, ville normalisée.
- Rejeter et journaliser les enregistrements invalides dans une table d'erreurs.

4.1.3 Module de traitement

- Normaliser les intitulés de postes .
- Extraire automatiquement les compétences techniques et soft skills depuis le texte des offres.
- Calculer les agrégations analytiques : top compétences par région, par métier, par période.
- Produire des tables Gold prêtes à l'analyse (aucune transformation supplémentaire requise à l'affichage).

4.1.4 Module d'analyse et KPIs (peuvent être modifié)

Les indicateurs clés de performance suivants doivent être calculés et accessibles :

Catégorie	KPI	Granularité
Tendances de compétences	Top N compétences les plus demandées	Par semaine / mois / trimestre
Tendances de compétences	Évolution temporelle d'une compétence	Variation en % sur période
Tendances de compétences	Compétences émergentes (croissance rapide)	Détection automatique
Tendances de compétences	Compétences en déclin	Détection automatique
Géographie	Répartition des offres par ville / pays	Carte choroplèthe
Géographie	Comparaison Maroc / France	Par compétence, par métier
Métiers	Top métiers par volume d'offres	Par région et période
Métiers	Prérequis de compétences par métier	Profil moyen du poste

Marché	Volume total d'offres collectées	Par source et période
Marché	Niveau d'expérience demandé	Répartition junior / mid / senior
Marché	Types de contrats (CDI, CDD, freelance)	Par pays et métier

4.1.5 Module AI Agents

- Un agent doit pouvoir répondre à des questions en langue naturelle sur les données Gold (ex. : “quelles sont les compétences les plus demandées à Casablanca ce mois-ci ?”).
- L'agent doit détecter les anomalies (pic soudain d'une compétence, absence de données d'une source) et envoyer des alertes.

4.1.6 Module de visualisation

- Tableaux de bord interactifs avec filtres sur : période, région, métier, compétence.
- Graphiques de tendances temporelles, cartes géographiques, nuages de mots de compétences.
- Accessibilité via navigateur web sans installation client.

4.2 Contraintes fonctionnelles

- Les données affichées dans les dashboards ne doivent pas avoir plus de 24h de retard.
- Toutes les données collectées doivent être conservées au minimum 24 mois.
- L'architecture doit garantir la reproductibilité des traitements (idempotence des pipelines).

5. Spécifications techniques

5.1 Architecture générale

L'architecture suit le paradigme Lakehouse Cloud Native, structurée en couches logiques indépendantes et communiquant via des interfaces standard. Elle est conçue pour être déployée sur Kubernetes et migrable vers Databricks sans modification du code métier.

Les cinq couches principales de l'architecture sont :

- Couche de collecte : scrapers et producers Kafka.
- Couche de streaming : Kafka topics, consumers, validation de schéma.
- Couche Lakehouse : stockage objet (MinIO / S3-compatible), tables Delta Lake en trois niveaux (Bronze, Silver, Gold).
- Couche de traitement et transformation : Apache Spark / PySpark pour le traitement lourd, dbt pour les transformations SQL Silver → Gold, pour l'extraction de compétences.
- Couche d'exposition : Trino / DuckDB pour les requêtes, Superset / Grafana pour la visualisation, API REST pour l'accès programmatique.

5.2 Stack technologique

5.2.1 Stratégie de développement et de migration

Le projet est développé avec un stack 100 % open source sur Kubernetes. L'architecture est conçue dès l'origine pour être compatible avec Databricks : l'ensemble du code métier (PySpark, dbt, LangChain, Kafka) reste identique lors d'une migration éventuelle. Seuls les fichiers de configuration (endpoint de stockage, adaptateur dbt, credentials) changent.

Couche	Open source (dev)	Migration Databricks	Portabilité
Collecte	Scrapy, Playwright	Inchangé	100 %
Streaming	Apache Kafka (KRaft)	Confluent Cloud (opt.)	100 %
Stockage objet	MinIO (API S3)	DBFS / S3 / ADLS	1 variable de config
Format tables	Delta Lake open source	Delta Lake natif	100 %
Traitement	PySpark (local / k8s)	Databricks Spark	100 %
Transformation SQL	dbt-spark / dbt-duckdb	dbt-databricks	1 ligne profiles.yml
NLP	spaCy, HuggingFace	Inchangé	100 %
AI Agents	LangChain / LlamaIndex	Inchangé	100 %
Query layer	Trino / DuckDB	Databricks SQL	Adaptation légère
Dashboards	Apache Superset, Grafana	Inchangé	100 %
Orchestration	Apache Airflow	Databricks Workflows	Adaptation légère

Conteneurs	Kubernetes + Helm	Inchangé (pour comp. ext.)	100 %
CI/CD	GitHub Actions + Docker	Inchangé	100 %

5.2.2 Principes d'architecture

- Aucune dépendance à un fournisseur cloud spécifique dans le code métier.
- Les pipelines sont idempotents : une exécution multiple du même job produit le même résultat.
- La configuration de chaque composant est externalisée dans des variables d'environnement ou des ConfigMaps Kubernetes.

5.3 Modèle de données

5.3.1 Couche Bronze (données brutes)

Table principale : offres_brutes

- Champs : exemples : id_hash, source, pays, ville_brute, titre_brut, description_brute, competences_brutes, date_publication, date_ingestion, type_contrat_brut, experience_brut, salaire_brut, entreprise.
- Partitionnement : par date_ingestion et par source.
- Pas de transformation — données exactement telles que collectées.

5.3.2 Couche Silver (données nettoyées)

- Normalisation des villes, des types de contrat, des niveaux d'expérience.
- Déduplication des offres (clé : id_hash).
- Extraction des compétences sous forme de tableau (array de strings normalisés).
- Détection de la langue de l'offre.

5.3.3 Couche Gold (données analytiques)

- Table : top_competences — agrégation des compétences par région, métier, période.
- Table : tendances_temporelles — évolution hebdomadaire / mensuelle des compétences.
- Table : profils_metiers — profil type de compétences par intitulé de poste.
- Table : comparaison_geo — comparatif Maroc vs France par compétence et métier.
- Table : marche_emploi_summary — KPIs généraux (volume, types de contrats, niveaux).

6. Ressources

6.1 Ressources techniques

6.1.1 Environnement de développement

- Machine locale ou serveur (pour Spark local + Kafka + k8s en mode minikube).
- Docker Desktop pour l'exécution locale des conteneurs.
- Minikube ou k3s pour le cluster Kubernetes local.
- Git + GitHub pour le versionnement du code.

6.1.2 Sources de données

- Rekrute.com — scraping (Maroc).
- Emploi.ma — scraping (Maroc).
- Welcome to the Jungle — scraping (France).
- Adzuna API — API publique avec clé gratuite (France, Maroc partiel).
- API Emploi (Pole Emploi / France Travail) — API officielle avec inscription.

6.1.3 Cloud et licences

- Databricks Community Edition — optionnel, pour validation de compatibilité et démo.
- GitHub Actions — 2 000 minutes/mois gratuites (suffisant pour CI/CD du projet).
- Toutes les technologies du stack open source sont gratuites et sans licence commerciale.

6.3 Ressources financières

Le projet est conçu pour être développé sans coût infrastructure significatif. L'ensemble du stack open source tourne localement ou sur des tiers gratuits. Un budget optionnel de l'ordre de 20 à 50 USD peut être envisagé pour des proxies rotatifs lors de la phase de collecte intensive, ou pour des clés API LLM (Claude API, OpenAI) si les agents IA n'utilisent pas un modèle local (Ollama).

7. Délais et planification

7.1 Approche

Le projet est organisé en 10 épics (EPIC 0 à EPIC 9), chaque épïc correspondant à un sprint de travail. Les durées indiquées ci-dessous sont des estimations indicatives à ajuster avec l'encadrant selon le calendrier académique.

Épic	Intitulé	Sprint	Durée est.	Livrables clés
EPIC 0	Cadrage et conception	Sprint 1	1 semaine	Cahier des charges, MCD, architecture
EPIC 1	Collecte de données (scraping + APIs)	Sprint 2	2 à 3 semaines	Scrapers opérationnels, producers Kafka
EPIC 2	Streaming et ingestion temps réel	Sprint 3	1 à 2 semaines	Kafka configuré, Bronze alimenté
EPIC 3	Lakehouse (Delta Lake / stockage)	Sprint 4	1 à 2 semaines	Couches Bronze, Silver, Gold initialisées
EPIC 4	Traitement et transformation (Spark + dbt)	Sprint 5	2 à 3 semaines	Pipelines Spark, modèles dbt, NLP
EPIC 5	AI Agents	Sprint 6	2 à 3 semaines	Agent LangChain opérationnel, rapports auto
EPIC 6	Query Layer	Sprint 7	1 semaine	Trino / DuckDB + API REST
EPIC 7	Dashboards	Sprint 7-8	1 à 2 semaines	Superset / Grafana avec tous les KPIs
EPIC 8	Cloud Native (Kubernetes)	Transversal	Tout au long du projet	Helm charts, CI/CD, monitoring
EPIC 9	Documentation et soutien	Sprint final	1 à 2 semaines	README, rapport, démo live

La durée totale estimée du projet est de 14 à 20 semaines selon l'intensité de travail et les contraintes académiques.

7.2 Jalons principaux

- Jalon 1 — Fin EPIC 0 : validation du cahier des charges et de l'architecture par l'encadrant.
- Jalon 2 — Fin EPIC 2 : premier flux de données de bout en bout (collecte → Kafka → Bronze).
- Jalon 3 — Fin EPIC 4 : pipeline complet de la collecte jusqu'aux tables Gold.
- Jalon 4 — Fin EPIC 7 : plateforme complète avec dashboards et agents IA.
- Jalon 5 — Fin EPIC 9 : soutien avec démonstration live.

8. Risques et mesures d'atténuation

8.1 Risques identifiés

Blocage des scrapers	Rotation de proxies, limitation du taux de requêtes, use d'APIs officielles en priorité.
Données insuffisantes pour le Maroc	Couverture multi-sources (Rekrute + Emploi.ma + LinkedIn Maroc), collecte plus fréquente.
Ressources compute insuffisantes en local	Utilisation de DuckDB pour les développements légers, Spark uniquement pour les traitements de masse.
Dépassement du quota Databricks Community	Architecture conçue pour tourner complètement en open source, Databricks reste optionnel.
Dérive du périmètre (scope creep)	Roadmap figée en début de projet, toute extension documentée comme backlog.
Qualité des données (bruit, doublons)	Pipeline de validation strict en Bronze, tests dbt en Silver, monitoring de qualité.

Annexes

Annexe A — Glossaire

Lakehouse	Architecture combinant la flexibilité du Data Lake et les performances du Data Warehouse, basée sur des tables Delta Lake.
Bronze / Silver / Gold	Nomenclature des couches de stockage : Bronze = données brutes, Silver = données nettoyées, Gold = données analytiques prêtes à l'usage.
Kafka	Plateforme de streaming d'événements distribuée, utilisée pour l'ingestion en temps réel.
dbt	Outil de transformation SQL (Data Build Tool) qui gère le lineage, les tests et la documentation des modèles de données.
AI Agent	Programme autonome basé sur un LLM capable d'effectuer des actions (requêtes, génération de rapports) en réponse à des objectifs.
Kubernetes	Système d'orchestration de conteneurs Docker, base de l'approche Cloud Native.
Cloud Native	Approche de conception d'applications conçues pour s'exécuter sur des infrastructures cloud (conteneurs, microservices, orchestration).
Databricks	Plateforme unifiée d'analytics et d'IA basée sur Apache Spark, proposée en version Community gratuite.
MinIO	Serveur de stockage objet open source compatible avec l'API Amazon S3, utilisé en remplacement local de S3.

Annexe B — Sources de données envisagées

Rekrute.com	Maroc — scraping — principale plateforme d'emploi marocaine
Emploi.ma	Maroc — scraping — annonces généralistes
Welcome to the Jungle	France — scraping — offres tech françaises
Indeed	Maroc + France — scraping — couverture internationale
Adzuna API	France + international — API publique gratuite
France Travail API	France — API officielle (Pole Emploi) — inscription gratuite